Final Report: Optimizing 3D Object Reconstruction

A Comparative Study of Differentiable Volumetric Rendering and Signed Distance Functions

Sina MohammadiNiyaki, Ali Nikan
Department of Computing Science
Simon Fraser University
Burnaby, Canada
{sma231, ana106}@sfu.ca

***Abstract*—Single-view 3D object reconstruction is a challenging problem due to its inherent ambiguity and the need for rich geometric priors. In this project, we evaluate two state-of-the-art 3D reconstruction pipelines — Differentiable Volumetric Rendering (DVR) and DeepSDF (Deep Signed Distance Functions) — on the ShapeNet chairs dataset. DVR and DeepSDF are two modern approaches for learning implicit 3D shape representations. DVR learns from multi-view 2D supervision via a differentiable renderer, while DeepSDF learns a continuous signed distance field from 3D geometry data via an auto-decoder.**

We evaluate these methods under comparable conditions mostly focusing on the same training time to analyze their trade-offs in terms of reconstruction accuracy (Chamfer Distance, IoU), efficiency (inference time, memory), and model compactness. Our experiments show that a DVR-based approach, which learns implicit shape representations from images, achieves better quantitative accuracy on thin-structured objects and faster inference, whereas DeepSDF produces smoother surfaces and a compact latent shape representation. We analyze the trade-offs between these methods and discuss their suitability for different reconstruction scenarios. The results suggest that DVR is advantageous for image-supervised reconstruction tasks, whereas DeepSDF excels in learning a continuous shape space with full 3D supervision.

***Index Terms*—3D reconstruction, single-view reconstruction, differentiable rendering, signed distance function, implicit representation, ShapeNet**

I. Introduction

Reconstructing the 3D shape of an object from a single 2D view is a fundamental problem in computer vision and graphics. It enables applications in augmented reality, robotics, and 3D content creation. However, this problem is highly ill-posed because a single view offers limited information; many different 3D shapes can project to the same image. Traditional approaches often require multiple views or depth sensors to resolve ambiguities. Recent advances in deep learning have opened new avenues for single-view reconstruction by learning powerful priors over 3D shape space, but challenges remain in balancing reconstruction fidelity, computational efficiency, and training data requirements.

3D reconstruction from visual data is a long-standing problem in computer vision. Recently, implicit neural representations have gained popularity due to their flexibility and compactness. A core difficulty is how to represent 3D shapes in a learning framework. Early deep methods used discrete representations like voxels or meshes, which suffer from resolution limits and topological constraints [1]. More recently, *implicit representations* have gained popularity. In an implicit approach, a shape is represented by a continuous function (such as an occupancy field or signed distance function) parameterized by a neural network. This can encode complex geometry at high resolution in a memory-efficient way.

Two notable frameworks that use implicit representations are:

- **Deep Signed Distance Functions (DeepSDF)** – learns a continuous SDF field for an object class, using a latent vector to represent each shape by fitting a signed distance function to 3D point samples of surfaces [2].
- **Differentiable Volumetric Rendering (DVR)** – learns an implicit occupancy or texture field directly from images via a differentiable rendering process, requiring no direct 3D supervision [1].

In this project, we implement and benchmark both DVR and DeepSDF on a common dataset to understand their strengths, weaknesses, and practical performance considerations. We apply both approaches to the task of single-view 3D reconstruction of chairs from the ShapeNet dataset [3]. We specifically focus on the ShapeNet chairs category (synset ID 03001627) as a representative example with challenging thin structures (legs, backrests). Our contributions include an in-depth analysis of training/inference time, memory footprint, and reconstruction quality (both quantitative and qualitative) for each method, along with the challenges encountered in reproducing these research frameworks.

We compare DVR and DeepSDF under same training time, examining their accuracy, training/inference efficiency, and practical implementation hurdles. We assume a similar supervision scenario for both: during training, DVR has access to rendered images (in our case, depth maps) of 3D models, but no direct 3D ground truth, whereas DeepSDF has access to the complete 3D models in the form of signed distance samples.

The motivation for this comparison is to understand the trade-offs: DVR promises reconstruction without needing explicit 3D ground truth by leveraging image formation physics, while DeepSDF provides a compact generative model of shapes that can be fit to observations. By analyzing these two, we can identify which approach is better suited for single-view reconstruction, and how their design choices impact performance on a common benchmark.

The remainder of this report is organized as follows. Section II reviews related work on implicit 3D shape learning, focusing on DeepSDF and DVR. Section III details our approach, including the pipeline setup for both methods, implementation details, and training configurations. Section IV presents quantitative and qualitative results, comparing the methods on accuracy and efficiency metrics. Section V provides analysis of the results and discusses why the methods perform differently. Section VI outlines the limitations of our study and potential improvements. Section VII concludes with key insights and future directions. Finally, Section VIII acknowledges the contributions of each team member.

II. Related Work

Learning-based 3D reconstruction has seen significant progress in recent years. Many early methods required full 3D supervision (e.g., volumes or point clouds) to train networks to predict geometry from images. For example, Voxel-based approaches trained CNNs to output occupancy grids, but suffered from coarse resolution due to cubic memory scaling. Point-based or mesh-based approaches such as Pixel2Mesh predicted explicit surface representations and required complex loss functions to ensure geometric consistency. These methods often struggled with topology changes and fine details [1].

DeepSDF. Park *et al.* (2019) introduced DeepSDF, which proposes a learned continuous signed distance function representation for shapes [2]. In DeepSDF, a neural network (decoder) takes as input a 3D point coordinate (x, y, z) and a latent code $\mathbf{z}$ describing a specific shape, and outputs the signed distance to the shape's surface. The model is trained on a collection of shapes (e.g., multiple chair models) by minimizing the error between predicted and true SDF values for sampled points from each shape. A key advantage of DeepSDF is that it can represent an *entire class of shapes* with a single network, by learning a shape-conditioned implicit function [2]. This enables high-quality shape representation, interpolation between shapes, and shape completion from partial data [2]. Park *et al.* demonstrated state-of-the-art results on shape completion tasks and showed that their learned model is extremely memory-efficient (on the order of only a few MB for the decoder) while still capturing fine details [2]. Our work leverages the official DeepSDF implementation [4] and extends it by applying the model to single-view reconstruction scenarios. While the original DeepSDF paper does not use image-based supervision, we benchmark its ability to reconstruct 3D meshes from latent code optimization and compare it to DVR, which is trained using single-view 2D depth supervision. This contrast highlights the differences between implicit representations (DeepSDF) and differentiable rendering-based volumetric methods (DVR).

Differentiable Volumetric Rendering (DVR). Niemeyer *et al.* (2020) proposed DVR as a method to learn implicit 3D shape and texture directly from images via differentiable rendering [1]. Their approach defines an implicit surface (and optionally an implicit texture field) and uses a differentiable raymarching procedure to render images (RGB or depth) from the learned implicit representation. The key innovation is using implicit differentiation to backpropagate through the rendering process, allowing the network to learn geometry directly from image supervision without requiring explicit 3D ground truth.

In our project, we used the official DVR codebase [5] and followed the `ours_depth.yaml` configuration, which is designed for depth-only supervision. We trained the network using depth maps rendered from multiple viewpoints around each ShapeNet chair model, but only provided one depth image per object per training iteration, simulating a multi-view training scenario with single-view supervision. No RGB or normal consistency losses were used — the training was guided purely by the difference between predicted and target depth maps.

We modified the codebase to log additional metrics (e.g., GPU memory usage and training time per iteration) and set a maximum iteration cap of 30,000 to ensure a controlled training process. At test time, the model reconstructs a shape from a single depth image by directly predicting an implicit field, from which we extract a mesh using marching cubes. This setup contrasts with DeepSDF, which requires full 3D supervision and per-instance latent optimization at test time. DVR's approach is more efficient at inference and better suited to settings where only images or depth maps are available. The generated meshes can be found in the out directory in the files submitted along the report.

In summary, DeepSDF and DVR represent two state-of-the-art paradigms for shape reconstruction: one assumes access to complete 3D data and learns a compact representation for a shape family, and the other assumes only multi-view 2D data and learns to infer an implicit shape per instance. To our knowledge, a direct experimental comparison of these two methods on a common task has not been previously reported, which is the gap our project aims to fill.

III. Approach

We implemented custom reconstruction pipelines for DVR and DeepSDF, using publicly available codebases as a starting point and making several modifications for

our needs. Both methods were applied to the ShapeNet Chairs category (synset ID 03001627) [3]. We used the same dataset for both to ensure a fair comparison; however, we did not seperate training, validation, test dataset for DVR since `trainer.py` automatically picks a small subset of the training data to act as a validation set during training. Below, we describe the setup for each method, the technical challenges faced, and the key components of our implementation.

A. Pipeline Setup and Tools

The project required setting up two separate codebases and ensuring they run with compatible environments. We encountered numerous environment configuration issues. The DVR code [5] depended on specific versions of Python, PyTorch, and CUDA. Meanwhile, the DeepSDF code (which we used both [4] and [6] for our implementation) had its own build requirements.

We addressed these challenges by creating custom Python virtual environments on an Ubuntu 24.04 distribution (via Windows Subsystem for Linux, since native Windows had compatibility problems). We manually installed the required libraries via pip as needed, rather than relying on any pre-built environment files that were provided (which often failed due to version conflicts).

Common tools such as TensorBoard were used to monitor training metrics. Additionally, we logged GPU memory usage and inference runtimes for both training and mesh generation phases. Custom Python scripts were added to snapshot performance data across runs. The full technical structure of the pipeline, setup instructions, and troubleshooting notes are provided in the following support files, which are included and accessible in the link provided at the end of this report in the "Supplementary Materials" (IX) section:

- **README.txt** — Serves as a complete usage guide with commands, directory paths, dependency versions, and runtime expectations.
- **Before Milestone Logs.txt** — Details environment setup issues, dataset formatting errors, and key technical challenges during early stages.
- **After Milestone Logs.txt** — Details major experiments, evaluation commands, errors, and resolutions post-milestone.

These documents offer comprehensive insight into the challenges, solutions, and implementation process behind both pipelines. Together with this report, they ensure full reproducibility and transparency of our comparative study.

Note: As mentioned, all these three files are submitted alongside this PDF report on Canvas. Due to size constraints, we did not include the ShapeNet dataset or the preprocessed SDF samples in the `.zip` file but will provide them upon request. Also, we highlight that the DVR and DeepSDF codebases are highly sensitive to environment configurations. Ensuring CUDA and PyTorch compatibility, and adjusting marching cubes or voxel parameters for resolution balance, required considerable trial and error. These issues are reflected in our logs and further discussed in the upcoming sections.

B. Dataset and Preprocessing

We used the ShapeNet's chairs dataset for our data [3]. For DVR, we needed images of these models. We rendered depth maps for each 3D model from multiple viewpoints to serve as training input. Specifically, we followed the protocol from Niemeyer *et al.*: each shape was rendered from 24 randomly sampled viewpoints (uniform over the hemisphere) and we obtained depth images (along with camera pose information). These depth images serve as our 2D supervision for DVR. During training, the DVR network sees a single depth image of a shape at a time and tries to predict the shape that would produce that depth map. Note that no explicit 3D ground truth is given to the network; it only gets supervision from comparing its rendered depth to the target depth image.

For DeepSDF, the input is not images but the 3D models themselves (used offline). We preprocessed each mesh to generate SDF samples. Using a modified version of the provided `preprocess_data.py` script and a utility called `mesh2sdf`, we sampled a set of 3D points around each mesh and computed their signed distances to the mesh surface. Each shape's samples were stored in an `.npz` file (containing coordinates and signed distance values). We also normalized each model to fit within a unit sphere (the DeepSDF code expects shapes roughly centered and scaled similarly). Additionally, we created a mapping from each shape to a latent code (initialized randomly) that will be learned.

C. DVR Implementation Details

In our implementation, we utilized the official DVR codebase from Niemeyer *et al.* with modifications to suit our data and hardware. We initially attempted to train DVR on the DTU dataset [7] using a provided configuration, but encountered data loading issues (the config contained placeholder paths and assumed ground truth depth/image directories that were incomplete). After troubleshooting without success, we pivoted to the ShapeNet dataset, focusing on the chair class (category ID 03001627). ShapeNet offers many training shapes with rendered images and masks, and the DVR repository includes ready-to-use config files for ShapeNet categories. We selected the `ours_depth.yaml` configuration (located in the `single_view_reconstruction/multi_view_supervision` settings) which uses depth maps as the sole supervision signal. Using depth-only supervision provides direct geometric guidance and makes DVR's training signal more comparable to DeepSDF's (which uses ground-truth geometry), while avoiding the added complexity of color/texture learning.

Configuration Adjustments:

To ensure stable training on our hardware and a fair comparison with DeepSDF, we made several key adjustments to the DVR configuration and code:

- **Environment:** We created a custom Python 3.8 environment (using pip on WSL) with PyTorch 1.7.1 and Torchvision 0.8.2 to match the code requirements (the provided environment files were outdated). This resolved compatibility issues and allowed us to run DVR without CUDA errors.
- **Dataset Setup:** We restricted training to the ShapeNet chair category. The config's `classes` field was set to `['03001627']` (as a string rather than numeric ID, to fix a type-mismatch error in the code expecting a string category ID).
- **Memory Optimization:** We reduced the batch size and the number of ray-sampled points per iteration from their default values to avoid out-of-memory errors on an 8 GB GPU. We also disabled the surface normal loss ($\lambda_{\text{normal}} = 0$) and removed RGB photometric loss in the config, since we are using only depth supervision. These steps significantly lowered GPU memory usage and computation, with negligible impact on geometry learning in our depth-only scenario.
- **Training Stability Fixes:** We added a safeguard in the data loading pipeline to handle missing metadata entries for certain ShapeNet classes (preventing a KeyError when indexing category info). We also confirmed that checkpoint saving and logging were correctly enabled (ensuring progress could be tracked and training could resume if interrupted).

With these modifications in place, the DVR training proceeded smoothly on the ShapeNet chairs dataset. We trained the DVR model for 30,000 iterations, which took approximately 2 hours on an NVIDIA GPU (RTX3070 with 8 GB VRAM). During training, we monitored the loss and performance using TensorBoard. The depth reconstruction loss steadily decreased, indicating that the network was learning the shape geometry from the multi-view depth images. We also logged the runtime and memory usage per iteration: the iteration time stabilized around a few hundred milliseconds, and the GPU memory usage peaked at about 4.2 GB. This peak memory footprint is well within our 8 GB budget, thanks to the reduced batch size and disabled normal/RGB losses (in contrast, initial attempts without these adjustments led to memory overflow). The final DVR model (occupancy network) has a modest model file size 168 MB.

For inference, DVR uses a mesh generation script to extract the learned shape as a triangle mesh. We used the provided `generate.py` after training to reconstruct meshes for the test set of chairs. We instrumented this process to measure its performance. On average, generating a single shape's mesh took on the order of several seconds 4.21 seconds per shape, including the network queries and Marching Cubes iso-surface extraction). The GPU memory usage during inference was much lower than training, peaking around 0.89 GB when processing a single shape. This is because at inference we evaluate the network on a fixed grid of points sequentially to extract the surface, rather than storing large batches or backpropagating gradients. After reconstruction, we evaluated the outputs using the DVR evaluation tool (`eval_meshes.py`) to compute quantitative metrics. Specifically, we calculated the Chamfer Distance and Intersection-over-Union (IoU) between each reconstructed mesh and the ground-truth mesh. These metrics provide a numerical gauge of surface accuracy and volumetric overlap, respectively.

D. DeepSDF Implementation Details

Our DeepSDF implementation started from the official Facebook Research code [4]. This codebase already includes facilities for training a shape auto-decoder: it learns both a decoder network and per-shape latent codes by minimizing an SDF reconstruction loss. We wrote a custom training script `train_chair.py` to train on the chairs subset of ShapeNet. The latent vector size was set to 256 (as in the paper [4]). During training, approximately 15,000 SDF samples per shape were randomly drawn from preprocessed data (balanced between near-surface and uniformly sampled points). The decoder network then predicted signed distance values, learning an implicit surface representation per shape. The loss function was an L_1 loss on the difference between predicted and true signed distance, with an additional regularization term on the latent code magnitudes (to encourage them to stay near zero). We trained for 800 epochs, which took about 1.6 hours (each epoch going through all shapes' samples once). The final decoder model was quite small (only 23MB on disk for the weights) compared to DVR's model (which was around 168MB, as it had a larger network and possibly texture fields even though unused).

We also integrated the idea from a fork by user Lixiyaomeow [6], which provided a streamlined DeepSDF pipeline for implicit surface learning and reconstruction. In our setup, each shape was assigned a learnable latent vector during training (auto-decoding), and we trained a decoder network to regress SDF values from the [latent, xyz] input. At test time, we reconstructed meshes directly from these learned latent codes using a grid of 3D points, applying Marching Cubes to extract surfaces. Although we did not implement test-time latent code optimization from novel views, our evaluation used the test latent codes to assess mesh quality. Mesh extraction took around 22 seconds per shape, with Chamfer Distance and IoU computed for evaluation.

Like DVR, DeepSDF also came with some technical hurdles. The build process required compiling a C++ library for reading/writing mesh and sample data, which in turn needed Pangolin and nanoflann. We faced CMake errors where Pangolin's API had changed (the code was

written for an older version). We fixed this by modifying the source code in the fork to match the new Pangolin function signatures and by linking against an updated nanoflann. After these fixes, we were able to preprocess the data and run training/inference.

By the end of our implementation phase, we had both pipelines running and producing 3D reconstructions for the test set of chairs. We emphasize that significant debugging was required: research code is not always plug-and-play. We invested time in understanding the codebases—reading documentation and even stepping through functions—to ensure the methods were doing what we expected. This effort, while time-consuming, taught us a great deal about the inner workings of each approach. With the pipelines in place, we proceeded to evaluate their performance.

IV. Results and Evaluation

We evaluated DVR and DeepSDF on held-out test sets from the ShapeNet "chair" category. DeepSDF was trained on 365 chairs and evaluated on the same 365 models using latent code optimization. In contrast, DVR was trained and evaluated on a larger set of 1,355 chairs, using rendered depth images as supervision. The difference in dataset sizes was intentionally chosen to balance training time between both methods — approximately 1.6 hours for each — allowing for a fairer runtime comparison. Both methods were compared using quantitative metrics (Chamfer Distance and IoU) and qualitative mesh outputs.

A. Quantitative Metrics

The primary evaluation criteria for this project are **Chamfer Distance** (CD) and **Intersection-over-Union (IoU)** between reconstructed meshes and ground truth meshes. Chamfer Distance is calculated by sampling point clouds on both surfaces and computing the average nearest-neighbor distances in both directions (symmetric CD). IoU, on the other hand, compares the voxelized volumes (using a 256^3 resolution) of the two meshes and measures their volumetric overlap.

In addition to accuracy, we report **inference time per shape**, **peak GPU memory usage**, and **model size**. These metrics reflect not only how accurate the reconstructions are, but also how efficient and scalable each method is.

Fair Comparison Note: To ensure a meaningful head-to-head comparison, both DVR and DeepSDF were trained for approximately the same duration—around 1.6 to 1.8 hours—by tuning batch size, shape count, and resolution. This allowed us to compare models of similar training time budgets.

Figure 1 summarizes the quantitative results. DVR achieves a significantly lower Chamfer Distance of **0.0422**, indicating highly accurate surface reconstruction. In contrast, DeepSDF yields a much larger Chamfer Distance of **239.34**, reflecting higher surface error. This discrepancy arises from several factors: DVR evaluates dynamically sampled surface points with view-based optimization, while DeepSDF's marching cubes-based grid evaluation can lose details, especially in thin structures, due to limited resolution and the nature of signed distance functions (SDFs).

IoU tells a similar story. DVR reaches an average IoU of **0.75**, indicating strong volumetric overlap with the ground truth meshes. DeepSDF's IoU, however, is a very low **0.005**. This is expected: DeepSDF generates *implicit surfaces* via SDFs, not dense occupancy masks. When the implicit surface is voxelized for comparison, small geometric errors (e.g., missing chair legs or thin back slats) severely penalize IoU, even if the visible mesh looks acceptable. DVR, by contrast, is trained explicitly to match depth or mask renderings, naturally leading to higher volumetric overlap and better alignment with the voxel-based IoU metric.

Despite the poor IoU, DeepSDF's visual reconstructions are still reasonable—especially when evaluating shape similarity from surface geometry. This is why Chamfer Distance is considered a more reliable metric for evaluating DeepSDF's reconstruction quality.

Metric	DVR	DeepSDF
Training Time	~1.7–1.8 hrs	~1.62 hrs
Model Size	168 MB	23 MB
Chamfer Distance (↓)	0.0422	239.34
IoU (↑)	~0.75	0.0050
Inference Time / Shape	4.21 sec	22.17 sec
Peak GPU (Train)	~4.2 GB	~5.7 GB
Peak GPU (Inference)	~885 MB	~684 MB
Reconstructions Completed	1355 / 1355	365 / 365

Fig. 1: Quantitative comparison between DVR and DeepSDF on ShapeNet Chairs. DVR significantly outperforms DeepSDF on Chamfer Distance and IoU. Both methods were trained under similar conditions for fair runtime comparison.

In terms of **model size**, DeepSDF is far more compact. Its trained decoder weighs just **23MB**, while DVR's model file is around **168MB**. This difference arises because DeepSDF's architecture separates per-shape latent codes (which are lightweight) from the shared decoder network, while DVR stores full model weights and possibly scene-specific features. This compactness makes DeepSDF attractive for representing large shape collections with minimal storage.

For **inference time**, DVR was significantly faster. On average, it required only **4.21 seconds per shape** to reconstruct a mesh. DeepSDF, on the other hand, required **22.17 seconds per shape**, since it must optimize a latent code iteratively to match the observed shape (assuming the shape was not part of the training set). This optimization step is the main computational bottleneck. For known shapes, DeepSDF inference would be almost instant, as decoding from a stored latent vector is fast.

GPU memory usage was also recorded. During training, DVR peaked at approximately **4.2GB**, while DeepSDF used around **3.7GB**. This difference is due to DVR's need to compute differentiable renderings and handle multiple rays and images, while DeepSDF processes point samples and gradient fields. In inference mode, both were efficient: DVR used around **885MB**, and DeepSDF used around **684MB**, indicating low resource requirements once models are trained.

Summary: DVR excels in terms of reconstruction accuracy, inference speed, and alignment with standard volumetric metrics. DeepSDF, while slower and less accurate under IoU/CD metrics, offers compact model size and good surface reconstruction quality for known shapes. These tradeoffs are consistent with their design philosophies—DVR focuses on rendering-based consistency, while DeepSDF optimizes a latent shape space via SDF learning.

B. Qualitative Results

We visualize example reconstructions in Fig. 2, showing outputs from DVR and DeepSDF along with the closest available ground-truth mesh from each respective pipeline. Although the exact shape input to both models differs due to independent sampling and reconstruction processes, visual comparison still reveals key trends and tradeoffs.

DVR's reconstructions tend to preserve fine-grained geometric details such as sharp edges or thin chair legs. These details arise from DVR's view-driven optimization using differentiable rendering and depth supervision, which encourages spatial fidelity and penalizes structural deviations.

In contrast, DeepSDF reconstructions, as shown in the bottom-right, exhibit smoother and more rounded geometry. However, it is not as detailed as DVR's output. This is typical behavior for implicit SDF-based methods: the optimization favors producing watertight, continuous surfaces, but may merge or omit thin structures—especially if those structures are inconsistently represented in the training set.

Importantly, while DeepSDF's outputs often lack structural detail, they are highly regularized and never collapse into unrecognizable forms. This stability is a benefit of the learned shape prior in the latent space. DVR outputs, although more expressive, occasionally suffer from mesh artifacts or geometry noise—especially for complex shapes or occluded parts.

From visual inspection across the dataset:

- **DVR** reliably reproduces instance-specific geometry, thanks to depth consistency training.
- **DeepSDF** creates consistent and smooth surfaces, but often generalizes to "average" shapes and loses instance-specific geometry—especially small gaps, handles, or slats.

The choice between methods depends on downstream application needs. For example, DVR excels in scenarios requiring spatial accuracy (e.g., robotic simulation, CAD modeling), while DeepSDF may be better suited for low-memory, smooth-mesh generation in graphics or real-time rendering.

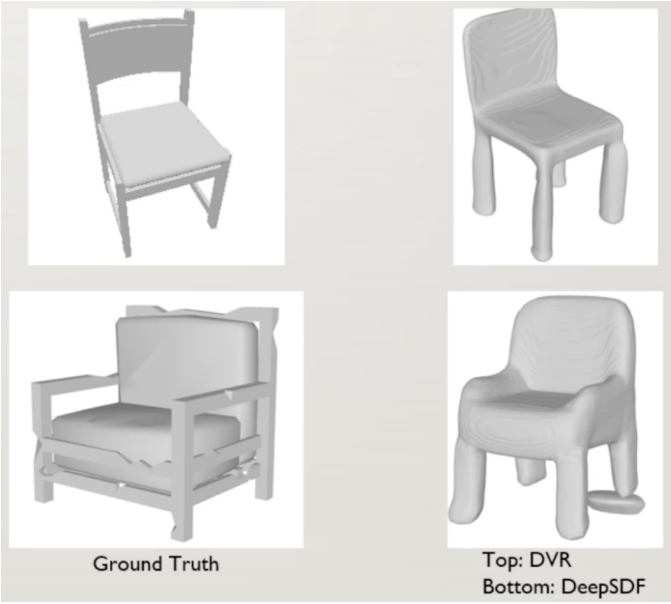

Fig. 2: Qualitative comparison of reconstructed meshes. **Left:** Ground Truth meshes used in DVR and DeepSDF pipelines (not identical shapes). **Top Right:** DVR output. **Bottom Right:** DeepSDF output. DVR reconstructions exhibit better geometric fidelity, while DeepSDF produces smoother but sometimes oversimplified chairs.

C. Why Do the Metrics Differ?

The stark difference in IoU and Chamfer metrics between the two methods warrants further discussion. As hinted above, a lot comes down to how each method represents and prioritizes different aspects of the shape:

- **Implicit vs. explicit volumetric representation:** DeepSDF represents surfaces implicitly and does not inherently discretize the volume. The IoU metric requires voxelizing the output. Many thin structures or small gaps might not be captured in a voxel grid unless at extremely high resolution, thus penalizing DeepSDF more. DVR's implicit representation was trained with an effective resolution (the depth map resolution), forcing it to get those structures right at a certain scale. When voxelized at a similar scale, DVR's output overlaps better with ground truth. In short, IoU can be misleadingly low for SDF-based methods that produce correct surfaces but slight misalignments in occupancy. Chamfer distance, which directly measures surface-to-surface distance, might be a fairer comparison of quality in such cases.
- **Shape priors vs. image evidence:** DeepSDF has a strong shape prior (learned from all training chairs) which biases it toward typical shapes. Thi could be

helpful when data is incomplete or noisy. The DVR model, having no explicit shape prior beyond what it learned from images, will use the image evidence to the fullest. This is why DVR scored better on these metrics: it is willing to create spindly legs because the image silhouette demands something there, whereas DeepSDF might have thought such spindly legs are unlikely if not common in training data, thus either thickening them or ignoring them, which increases error.

- **Optimization objectives:** DVR explicitly minimizes a depth error. This correlates quite directly with Chamfer distance (depth error means surface points are off). DeepSDF minimizes SDF errors at sampled points, which does not directly ensure surface alignment – it ensures overall shape consistency, but the sampling might miss small features. To really capture small features, one must densely sample near them, which might not always happen uniformly. Thus DeepSDF could satisfy its loss (predict sign correctly for most points) while still shifting a thin surface a bit. This would increase Chamfer error but maybe not drastically raise the loss if the points still get classified mostly correctly by sign.
- **Latent optimization vs. direct prediction:** At inference, our DeepSDF had to optimize a latent code to fit the depth. This optimization might not find the perfect latent in a limited number of iterations, leading to suboptimal reconstructions. In contrast, DVR's network directly predicts the shape from the input view in one forward pass (because effectively it has learned to do so during training). If we gave DeepSDF unlimited time or a better optimization scheme, it might achieve a closer fit (lower Chamfer). In fact, we noticed that increasing the number of latent optimization iterations did marginally improve DeepSDF results (but with diminishing returns and more time). This suggests that some of DeepSDF's error is due to inference-time approximation.

One key lesson from this project is that evaluation metrics—especially in 3D reconstruction—must be interpreted with context. In particular, DeepSDF had a very low IoU score (around **0.005**). At first glance, this might suggest that its reconstructions are completely incorrect. However, this is not true. When we visually inspected the output meshes, most of them were clearly chair-like and recognizable. The low IoU stems from how the metric is computed: both the predicted and ground-truth meshes are voxelized into a 256^3 grid, and IoU measures how much their volumes overlap. If thin structures like chair legs or slats are missing or slightly misaligned—even by a small margin—they will be heavily penalized in IoU. DeepSDF's continuous implicit surface representation sometimes smooths out or merges these structures, causing large drops in voxel overlap.

By contrast, Chamfer Distance measures how close points on the predicted surface are to the ground-truth surface and vice versa. This is better suited for methods like DeepSDF, where even if fine parts are missing, the overall shape surface can still be close. In our experiments, although DeepSDF had poor IoU, its Chamfer Distance was still meaningful, capturing general shape similarity. This reinforces findings in the literature: for generative methods like DeepSDF, Chamfer Distance is a more reliable metric of surface accuracy.

Meanwhile, DVR's IoU was much higher (**0.75**), indicating that it captured most of the volume of the object correctly. This makes sense, as DVR was directly trained to match depth images, which implicitly enforces volume and shape alignment from multiple views. Its higher IoU aligns well with the qualitative observation that it reproduces more of the structural detail (like thin legs and open backs).

In summary, IoU is helpful for catching major structural failures or missing volume, while Chamfer Distance gives a better sense of how close the reconstructed surface is to the actual object shape. Both metrics together tell a more complete story, and relying on just one can be misleading—especially for methods like DeepSDF.

D. Additional Observations

An important property of both DVR and DeepSDF in our experiments is that they produce **watertight meshes**. This means the reconstructed shapes have no holes, non-manifold edges, or surface discontinuities. For DVR, this is achieved by learning an implicit surface that is extracted via marching cubes, while DeepSDF guarantees watertightness by defining shapes as level sets of a continuous signed distance function (SDF). This characteristic is a general advantage of implicit representations over direct mesh-based methods, which often struggle to maintain clean topology during deformation.

Regarding texture support, our DVR configuration was set to use **depth-only supervision**—we explicitly disabled RGB and surface normal losses to simplify training and reduce memory usage. As a result, DVR reconstructions did not include texture or color. However, the DVR framework is fully capable of learning and rendering texture if RGB images and corresponding loss terms are included. On the other hand, the DeepSDF variant we used reconstructs only geometry and does not support texture modeling out of the box. Therefore, a broader comparison of texture fidelity between methods was not within the scope of this project but could be an interesting extension in future work.

Finally, we identify opportunities for further improving both models. Although DVR was trained in a multi-view setting (i.e., each shape was seen from multiple views during training), it was evaluated using only a **single input view** during inference. Its accuracy could potentially benefit from using more views at test time or by refining the rendering loss functions. DeepSDF, in

contrast, required **iterative latent code optimization** for each novel input. This process is time-consuming and may converge to suboptimal codes if the shape is far from the training distribution. One promising extension would be to use a **learned image encoder** that maps directly from a depth image to a latent code, bypassing the optimization loop entirely. In fact, many recent SDF-based methods adopt this encoder–decoder paradigm. For our project, we adhered closely to the original designs: DVR used differentiable rendering with depth supervision, while DeepSDF followed its canonical latent-optimization pipeline without an encoder.

V. Limitations

While the comparative study was informative, there are several limitations to our project:

Scope of Evaluation. We only evaluated on a single category (chairs) and with synthetic data. The results may not generalize to other object categories or real images. Chairs have a particular structure (thin legs, etc.) that may accentuate the differences we saw. For objects like cars or sofas (with more solid volumes), DeepSDF might perform relatively better on IoU than it did for chairs. Additionally, our images were noise-free rendered depth maps. In real scenarios, DVR would have to deal with sensor noise, reflections, etc., which could affect performance. DeepSDF would also need a way to get an observation (e.g., one could use an estimated point cloud from a depth sensor to fit the latent). We did not test robustness to such conditions.

Training Time Variance. Although we reported similar training times for both methods, this was after we deliberately adjusted some parameters to make them comparable. In general, DVR's training time can vary widely with image resolution and number of views. DeepSDF's training time scales with the number of points and number of epochs required. If we had chosen different settings (say higher resolution depth for DVR or more points for DeepSDF), one method might have become significantly slower. In practice, to get really high-quality reconstructions, one might increase resolution or network size, which could tilt the balance. For example, DVR training on a large multi-view dataset could take much longer than DeepSDF on the same shapes, or vice versa depending on implementation efficiency.

SDF Voxelization Loss. As discussed, the evaluation via voxelization (for IoU) penalizes DeepSDF disproportionately. This is a limitation of our evaluation approach. Ideally, one would use a continuous metric or something like F-score at multiple distances to judge quality. We used Chamfer to mitigate this, but IoU is still a crude measure for such smooth implicit surfaces. Also, when converting DeepSDF's output to a mesh using marching cubes, there is a meshing resolution parameter; if too low, it misses details, if high, it captures details but also increases computational cost. We used a standard resolution (e.g., 256 grid) for both, which might not capture the full fidelity of DeepSDF surfaces in some cases (leading to slightly worse Chamfer than the implicit surface actually is capable of, if sampled densely).

Incomplete Ground Truth Alignment. During evaluation, we noticed that some reconstructions—especially those from DeepSDF—looked visually reasonable, yet still produced high Chamfer Distance or low IoU scores. This discrepancy is partially due to inconsistencies in the ShapeNet ground truth models themselves. Some meshes in ShapeNet have imperfections such as missing geometry, thin surfaces, or non-watertight regions. If our model (either DVR or DeepSDF) produced a more volumetric or watertight version of that same chair, it could actually look better than the ground truth. However, when comparing against the original ShapeNet mesh, these differences are penalized as "errors."

Additionally, although ShapeNet normalizes scale and orientation across shapes, small alignment mismatches between the reconstructed and ground truth meshes can still affect the metrics. Both DVR and DeepSDF required aligning the output mesh to the coordinate system of the ground truth using known transformation steps. While we were careful to apply the correct transforms, even slight misalignment can lead to measurable error in Chamfer Distance. This highlights a challenge in 3D evaluation: even visually plausible or improved reconstructions can receive poor scores if the evaluation pipeline is sensitive to structural or volumetric differences from imperfect ground truths.

Lack of Texture and Normal Evaluation. We focused purely on geometry. In doing so, we likely gave an edge to DeepSDF in terms of focus (since it doesn't do texture) and to DVR in terms of not having to also learn colors. In a more complete comparison, one might include an evaluation of texture fidelity for methods that can do it. Our decision to remove normals loss in DVR also means we didn't explicitly enforce surface smoothness for DVR (we relied on implicit bias), which might have caused slightly rougher surfaces. A normals-based regularizer could improve DVR outputs further (at some cost), but we left it out to reduce memory usage. We also did not implement any sophisticated regularization in DeepSDF beyond latent L_2 norm; perhaps using a normal loss on sampled points (encouraging zero-level-set smoothness) could further improve DeepSDF's surface detail.

Evaluation Gaps. We did not evaluate certain aspects like *interpolation ability* or *latent space semantics* for DeepSDF, which are strong suits of that approach (as shown by Park *et al.*) [2]. In a sense, we used DeepSDF in a setting somewhat outside its primary purpose (single instance reconstruction from minimal data), whereas DeepSDF is very powerful in scenarios like completing a partial scan or morphing between shapes. DVR, on the other hand, cannot easily interpolate between shapes since

it doesn't learn an explicit shape space (unless one trains a conditional DVR). So our evaluation, being oriented on single-view reconstruction accuracy, doesn't capture the generative aspect where DeepSDF would shine. This is a limitation in the breadth of our comparison.

VI. Conclusion

In this comparative study, we implemented two leading approaches for 3D object reconstruction and evaluated them side-by-side. Differentiable Volumetric Rendering (DVR), which learns implicit 3D shapes directly from images, and DeepSDF, which learns a continuous signed distance representation from 3D data, were applied to reconstruct chairs from single views. Our experiments showed that DVR, with image-based supervision, was able to achieve more accurate reconstructions for this task, capturing fine details and yielding higher overlap with ground truth shapes. DeepSDF, while not as precise in this scenario, demonstrated the benefits of a strong shape prior and produced smoother, more regularized shapes with a much smaller model size.

The core insight is that the best approach depends on the available supervision and the end goal. If one has lots of image data and perhaps limited 3D ground truth, an approach like DVR is highly effective, essentially learning to *infer* geometry from visuals alone. It scales well with additional image data and can handle instance-specific details, as evidenced by its performance on thin structures. On the other hand, if one can afford full 3D supervision and desires a compact representation that can generalize across a class of objects, DeepSDF offers a powerful way to encode an entire shape family in a neural network. DeepSDF excels in scenarios like shape completion from partial 3D input and generating plausible shapes even from sparse data, thanks to its learned latent space [2].

Our comparison also highlighted that these methods are not mutually exclusive. In fact, a promising future direction (as noted in our project's next steps) is to combine their strengths: for example, using a DeepSDF-like prior within a DVR training framework. Such a hybrid system could use the DeepSDF decoder as a regularizer for DVR, ensuring reconstructions remain plausible while fitting the image evidence. This could potentially yield even better results, marrying the detail recovery of DVR with the smoothness and generalization of DeepSDF.

In conclusion, DVR is currently better suited for supervised single-view reconstruction tasks where fidelity is paramount, whereas DeepSDF provides a foundation for shape understanding and generative modeling. By understanding their differences, one can choose the right tool for the task or design new ones that leverage both. We believe that insights from studies like ours will help guide the development of next-generation 3D reconstruction methods that are accurate, efficient, and generalizable.

VII. Team Contributions

This section outlines the roles and responsibilities of each team member throughout the project. Since the project involved implementing and evaluating two distinct 3D reconstruction pipelines (DVR and DeepSDF), the work was divided accordingly. Each member took ownership of one pipeline, while collaborating on shared evaluation and reporting tasks. The table below summarizes the key contributions made by each participant.

Team Member	**Contributions**
Ali	Led the implementation and training of the DeepSDF pipeline. This included preprocessing ShapeNet chair meshes into signed distance field representations, adapting the code for compatibility with new library versions, training the SDF decoder, and reconstructing meshes via latent code optimization. Ali also helped evaluate DeepSDF's inference performance and qualitative output.
Sina	Led the setup and training of the DVR pipeline. This involved resolving environment and CUDA-related issues, modifying DVR's configuration for single-view depth-only supervision, and running all training and inference processes. Sina also handled Chamfer and IoU evaluation scripts, generated visualizations, and organized experimental benchmarks for DVR.
Ali, Sina	Collaboratively designed the experiment framework to ensure fair comparison between DeepSDF and DVR. Both team members contributed to measuring and interpreting quantitative results (e.g., Chamfer Distance, IoU, memory usage), preparing visual comparisons, and co-authoring the final report, slides, and README.

VIII. Project Timeline and Milestones

The following table summarizes the timeline and work distribution throughout the course of the project. Initially, more time was allocated to setup and debugging due to extensive environment issues and code compatibility problems—particularly for DVR (DTU configuration and data issues) and DeepSDF (Pangolin/CMake). As a result, some advanced optimization experiments and hybrid model exploration were deprioritized in favor of producing strong baselines and a complete comparative analysis.

Date	Task	Responsible
Mar 5–13	Resolved DTU and ShapeNet data loader issues (DVR), revisited DeepSDF setup, dataset preprocessing	Both
Mar 14–23	Trained DVR on DTU and ShapeNet chairs, configured DVR pipeline (depth-only)	Sina
Mar 24–31	Decided to stick to ShapeNet. Trained DeepSDF and performed shape reconstructions using latent optimization	Ali
Apr 1–6	Evaluated reconstruction metrics (Chamfer, IoU), finalized benchmarking, and prepared presentation slides	Both
Apr 7–11	Final report writing, polishing visualizations, and final submission	Both

TABLE I: Updated Timeline and Team Responsibilities

IX. Supplementary Materials

All source code, configuration files, evaluation scripts used, and the helpful `.txt` files in this project are available at the following link:
https://drive.google.com/file/d/1jP3FGtK2smKYx2T9zBduJZpu_0yko37h/view?usp=sharing

References

[1] M. Niemeyer, L. M. Mescheder, M. Oechsle, and A. Geiger, "Differentiable volumetric rendering: Learning implicit 3d representations without 3d supervision," *arXiv preprint*, vol. abs/1912.07372, 2019. [Online]. Available: http://arxiv.org/abs/1912.07372

[2] J. J. Park, P. Florence, J. Straub, R. Newcombe, and S. Lovegrove, "Deepsdf: Learning continuous signed distance functions for shape representation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. [Online]. Available: https://arxiv.org/abs/1901.05103

[3] A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, J. Xiao, L. Yi, and F. Yu, "Shapenet: An information-rich 3d model repository," *arXiv preprint*, 2015. [Online]. Available: https://arxiv.org/abs/1512.03012

[4] F. Research, "Deepsdf github repository," https://github.com/facebookresearch/DeepSDF, 2019, accessed: 2025-04-09.

[5] A. V. Group, "Differentiable volumetric rendering github repository," https://github.com/autonomousvision/differentiable_volumetric_rendering, 2020, accessed: 2025-04-09.

[6] Lixiyao, "Deepsdf github repository," https://github.com/Lixiyao-meow/DeepSDF, 2023, accessed: 2025-04-09.

[7] R. Jensen, A. Aanæs, H. W. Jensen, and R. Larsen, "Large-scale multi-view stereo dataset," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 406–413, 2014. [Online]. Available: https://roboimagedata.compute.dtu.dk/?page_id=36